

PECS — Persistent Engineering Continuity System

GitHub Release Documentation

Version: Experimental Prototype

License: User to determine before public release.

IMPORTANT WARNING

EXPERIMENTAL SOFTWARE

PECS was created through iterative AI-assisted development and architectural experimentation.

It is:

- experimental
- incomplete
- unverified
- not production-certified
- not security-audited
- not performance-audited
- not reliability-certified

This software is provided:

AS IS
WHERE IS
WITH ALL FAULTS

without any warranties or guarantees of any kind.

NO WARRANTY DISCLAIMER

The authors/contributors provide:

- no warranty
- no guarantee of correctness
- no guarantee of architectural stability
- no guarantee of data integrity
- no guarantee of compatibility

- no guarantee of fitness for any purpose

Use entirely at your own risk.

The system may:

- corrupt continuity assumptions
- generate incorrect locality inference
- produce incorrect architectural guidance
- increase project entropy
- degrade AI coding quality
- create duplicate implementations
- damage workspace structure
- produce incomplete or unstable retrieval behavior

Users are solely responsible for:

- backups
- testing
- validation
- sandboxing
- production safety
- security review
- code review
- architectural review

RECOMMENDED USAGE POLICY

PECS should initially ONLY be used:

- in isolated test workspaces
- on cloned projects
- with version control
- with frequent commits
- with human supervision

PECS should NOT initially be used:

- directly on production systems
 - on safety-critical systems
 - on financial systems
 - on medical systems
 - on legal automation systems
 - on security-sensitive infrastructure
-

WHAT IS PECS?

PECS stands for:

Persistent Engineering Continuity System

PECS is an experimental continuity-routing framework designed to reduce architectural entropy during AI-assisted software development.

It attempts to improve:

- coding continuity
- locality stability
- duplicate reduction
- architectural persistence
- compact context retrieval

especially during:

- long-running AI-assisted projects
- vibe coding workflows
- local-model development
- constrained-context workflows

THE PROBLEM PECS ATTEMPTS TO SOLVE

Modern AI coding systems frequently suffer from:

- architectural drift
- duplicate helpers
- scattered utilities
- continuity fragmentation
- wrapper proliferation
- locality loss
- repetitive rewrites
- broken assumptions
- context collapse

Especially near the final stages of projects.

In many cases the failure is NOT caused by inability to code.

Instead it is caused by:

continuity degradation over time

PECS attempts to reduce this degradation.

CORE DESIGN PHILOSOPHY

PECS is NOT intended to be:

- an autonomous coding agent
- a self-governing architecture system
- a semantic reasoning engine
- an orchestration AI
- an autonomous project manager

PECS is intended to be:

a deterministic continuity routing layer

Its primary role is:

- preserving locality continuity
 - reconstructing continuity state
 - compacting context
 - routing relevant engineering continuity
 - reducing duplicate evolution
-

FUNDAMENTAL DESIGN PRINCIPLE

PECS operates on the assumption that:

coding continuity is structural
not linguistic

Large language models are very effective at:

- language generation
- pattern completion
- code synthesis

but often weak at:

- long-term continuity preservation
- canonical ownership stability
- architectural locality persistence
- deterministic engineering evolution

PECS attempts to provide a continuity layer around the model.

PECS ARCHITECTURE OVERVIEW

PECS consists of:

Shared Continuity Infrastructure

Shared systems include:

- registry
- retrieval
- locality mapping
- object models
- workspace scanning

PECS-PRO

PECS-PRO is the richer continuity runtime.

Designed for:

- larger projects
- long-running projects
- online models
- larger context windows
- continuity reconstruction
- richer ingestion

PECS-PRO includes:

- ingestion
- incremental updates
- runtime overlays
- retrieval routing
- continuity runtime
- testing console

PECS-LITE

PECS-LITE is the lightweight continuity runtime.

Designed for:

- local models
- 8k / 16k / 32k contexts

- constrained inference
- minimal runtime overhead

PECS-LITE intentionally avoids:

- deep orchestration
 - semantic supervision
 - recursive governance
 - heavy reconstruction
-

FINALIZED PECS PRINCIPLE

PECS should tell the model:

- WHERE to look
- WHAT belongs together
- WHAT already exists
- WHAT outcome is expected

PECS should NOT tell the model:

- HOW to think
-

EXPECTED BENEFITS

Potential benefits include:

- reduced duplicate generation
- reduced locality drift
- reduced prompt reconstruction
- lower context usage
- more stable continuation behavior
- reduced wrapper proliferation
- more reusable implementations
- better long-session stability

Especially for:

- local models
 - vibe coding
 - long-running AI coding projects
-

IMPORTANT LIMITATIONS

PECS does NOT guarantee:

- correct architecture
- correct implementations
- perfect continuity
- duplicate elimination
- stable retrieval
- deterministic LLM behavior

PECS does NOT replace:

- human review
- software engineering discipline
- testing
- architecture planning
- debugging
- code review

PECS only attempts to:

reduce continuity entropy

CURRENT PROJECT STATUS

Current state:

experimental prototype

The architecture is still evolving.

Current implementation status:

- partially consolidated
- partially normalized
- not production-hardened
- not performance-tested
- not benchmarked
- not security-audited

Community testing and expert review are strongly encouraged.

DIRECTORY STRUCTURE — PECS PRO

```
pecs/  
  
  core/  
    registry.py  
    workspace_scanner.py  
    object_model.py  
  
  continuity/  
    object_engine.py  
    retrieval_engine.py  
  
  runtime/  
    final_runtime_core.py  
    runtime_cache.py  
    live_continuity_daemon.py  
    pecs_incremental_update_engine.py  
    pecs_runtime_overlay_manager.py  
  
  integrations/  
    continue_adapter.py  
  
  ingestion/  
    chat_ingestion.py  
  
  exports/  
    exporter.py  
  
  tools/  
    pecs_testing_console.py
```

DIRECTORY STRUCTURE — PECS LITE

```
pecs_lite/  
  
  core/  
    pecs_lite_registry.py  
    pecs_lite_object_model.py  
  
  runtime/  
    pecs_lite_runtime_core.py  
    pecs_lite_runtime_cache.py  
    run_lite.py
```

```
integrations/  
  pecs_lite_continue_adapter.py
```

INSTALLATION — PECS PRO

STEP 1 — CREATE TEST WORKSPACE

Recommended:

```
project_pecs_pro_test/
```

Do NOT initially integrate directly into production workspaces.

STEP 2 — EXTRACT PACKAGE

Extract PECS-PRO package into project root.

STEP 3 — CREATE REQUIRED FOLDERS

```
pecs/cache  
pecs/logs  
pecs/registry  
pecs/runtime/pro  
pecs/runtime/lite  
.continue/rules
```

STEP 4 — INITIALIZE DATABASE

Create:

```
pecs/registry/objects.db
```

Then run bootstrap SQL.

Example schema:

```
CREATE TABLE IF NOT EXISTS objects (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
object_id TEXT,  
problem_to_solve TEXT,  
expected_result TEXT,  
canonical_locality TEXT,  
recurrence_weight REAL DEFAULT 0,  
locality_confidence REAL DEFAULT 0  
);
```

Additional tables:

- object_locality
- object_relations
- incremental_file_state

STEP 5 — RUN RUNTIME

```
python pecs/runtime/final_runtime_core.py
```

Expected outputs:

```
pecs/logs/final_runtime.json  
pecs/cache/runtime_context.json  
.continue/rules/pecs_runtime.md
```

STEP 6 — TEST CONSOLE

```
python pecs/tools/pecs_testing_console.py
```

This allows inspection of:

- objects
- localities
- relations
- runtime continuity

INSTALLATION — PECS LITE

STEP 1 — CREATE TEST WORKSPACE

Recommended:

```
project_pecs_lite_test/
```

STEP 2 — EXTRACT PACKAGE

Extract PECS-LITE package into project root.

STEP 3 — RUN LITE RUNTIME

```
python pecs_lite/runtime/run_lite.py
```

CONTINUE INTEGRATION

PECS currently integrates primarily through:

```
.continue/rules/
```

Generated runtime continuity is exported into:

```
pecs_runtime.md
```

This allows injected continuity guidance for:

- Continue
 - Copilot workflows
 - AI-assisted coding sessions
-

RECOMMENDED DEVELOPMENT WORKFLOW

Recommended workflow:

1. Create feature
 2. Allow PECS runtime update
 3. Inspect continuity state
 4. Verify canonical locality
 5. Generate implementation through model
 6. Commit frequently
 7. Validate retrieval stability
-

WHAT PECS CURRENTLY DOES WELL

Current strengths:

- continuity routing
 - compact retrieval
 - locality stabilization
 - runtime overlays
 - deterministic structure
 - lightweight continuity persistence
-

WHAT PECS CURRENTLY DOES POORLY

Current weaknesses:

- automatic object extraction
- canonical inference
- relation quality
- continuity scoring
- retrieval tuning
- duplicate confidence inference

These areas require significant future work.

IMPORTANT ENGINEERING GUIDELINES

DO NOT

Do NOT allow PECS to:

- autonomously rewrite architecture
- autonomously delete code
- autonomously move implementations
- autonomously consolidate projects

PECS should remain:

continuity infrastructure

NOT:

an autonomous engineering authority

INITIAL EXPECTATION MANAGEMENT

PECS will likely initially:

- misclassify locality
- miss duplicate relations
- over-retrieve
- under-retrieve
- infer weak canonicals

This is expected.

The realistic goal is NOT:

perfect continuity

The realistic goal is:

reduced entropy growth

CONTRIBUTING

Community testing and expert contribution are strongly encouraged.

Areas requiring research include:

- deterministic object extraction
- locality inference
- duplicate detection
- continuity retrieval ranking
- retrieval compression
- architectural continuity metrics
- long-session continuity preservation

FUTURE DIRECTIONS

Potential future directions:

- VS Code extension
- retrieval visualization
- continuity graphs
- runtime dashboards
- improved object extraction

- retrieval ranking
- locality confidence models
- deterministic architectural indexing

These remain experimental concepts.

FINAL NOTE

PECS is an attempt to treat AI-assisted coding as:

a continuity preservation problem

rather than purely:

a language generation problem

Whether this approach is effective remains an open engineering question.

Community validation, criticism, experimentation, and redesign are encouraged.